Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 15: Inverted Pendulum Robot
Bi-Weekly Update 2

Moustafa Khalil, Katlynn Ly
Eric Chang, Gage Fox
Sponsor: Aniruddha Datta
TA: Sabyasachi Gupta

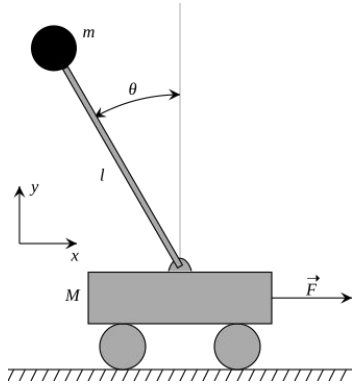
Project Summary

Problem statement:

- Autonomous warehouse robots generally experience instability when handling unbalanced loads or making sudden maneuvers. These movements can cause tilting, making balance a key challenge.

Our Solution Will:

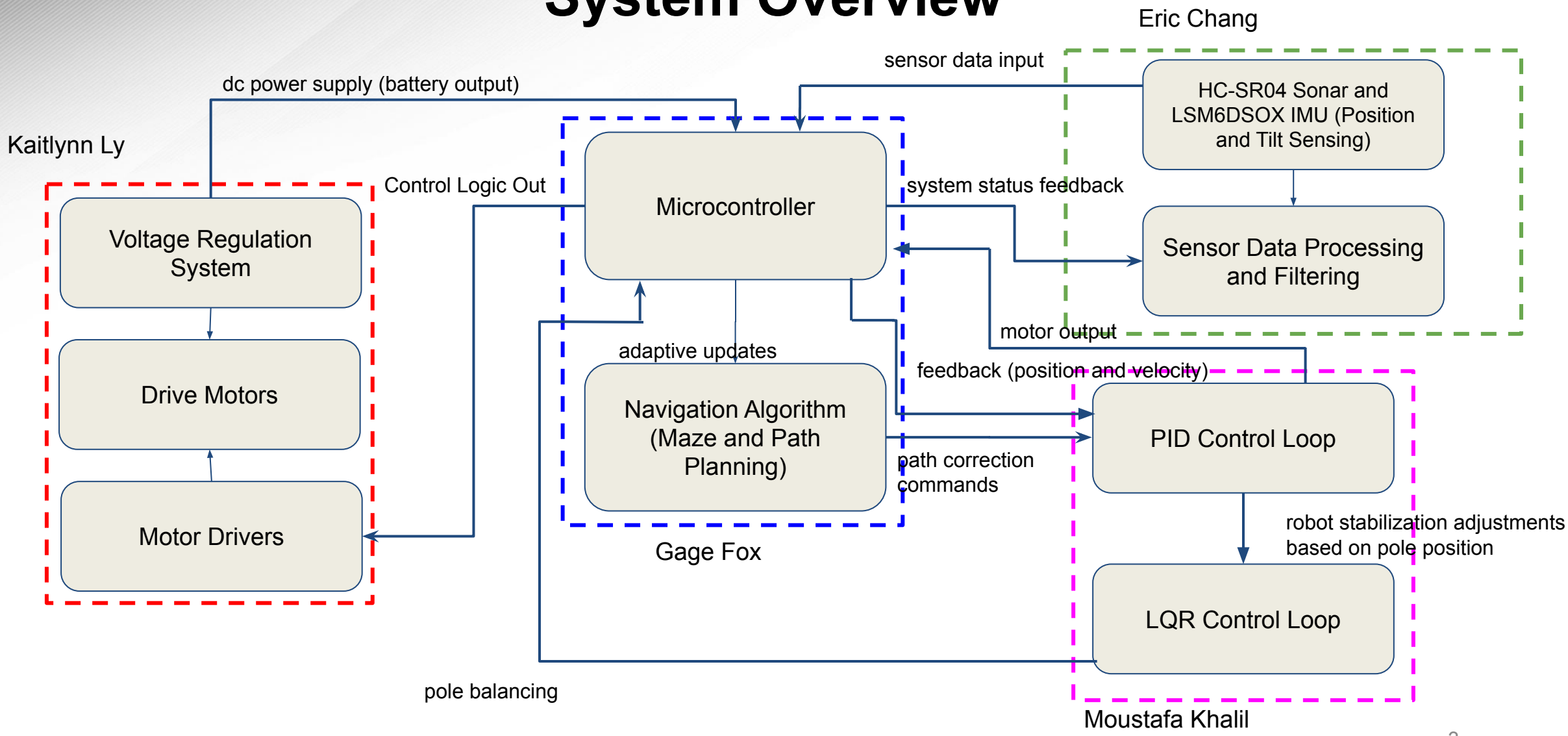
- Deliver an autonomous, self-navigating robot that maintains stability of an unstable payload while moving through constrained spaces.
- Integrate mechanical, sensing, power, and control subsystems into a cohesive, microcontroller-based platform designed for reliability and safety.



Side View

Isometric View

System Overview



Project Timeline

Subsystem Designs and Testing (complete by 2/9)	Integration of controls, sensors, and MCU (complete by 2/16)	Integration of MCU and motors (to complete by 2/20)	Full System Integration (complete by 3/6)	Full System Testing (complete by 3/20)	System Validation (complete by 4/1)	Demo and Final Report (complete by 4/20)

Controls Subsystem

Moustafa Khalil

Accomplishments since last update 13 hrs of effort	Ongoing progress/problems and plans until the next presentation
● Converted Code to C++ ● Calculated Inertia	● Integration with MCU

Subsystem 1

Moustafa Khalil

pidcplus > main()

```cpp
#include <iostream>
#include <Eigen/Dense>
#include <complex>

using namespace Eigen;



int main() {
    // ---------------------------------------------------
    // 1. System Parameters
    double M = 3.5;
    double m = 1.49;
    double L = 0.505;
    double r = 0.0355;
    double g = 9.81;
    double I = ;

    double den = I * (M + m) + m * M * std::pow(L, 2);

    // 2. State Space Matrices (A, B, C, D)
    MatrixXd A(4, 4);
    A << 0, 1, 0, 0,
         0, 0, -(std::pow(m, 2) * g * std::pow(L, 2)) / den, 0,
         0, 0, 0, 1,
         0, 0, (m * g * L * (M + m)) / den, 0;

    VectorXd B(4);
    B << 0,
         (m * std::pow(L, 2) + I) / (den * r),
         0,
         -(m * L) / (den * r);

    RowVectorXd C(4);
    C << 1, 0, 0, 0; // Controls Position (State 1)
```

```cpp
// 3. Defining the LQR Inner Loop (Stabilization)
RowVectorXd K_LQR(4);
K_LQR << 0, 0, 25.3830, 2.8;

// 4. the "Stabilized Plant"
// A_closed = A - B * K_LQR
MatrixXd A_closed = A - B * K_LQR;

// 5. PID Calculation

std::cout << "-----------------------------------------" << std::endl;
std::cout << "STABILIZED PLANT MATRIX (A_closed):" << std::endl;
std::cout << A_closed << std::endl;
std::cout << "-----------------------------------------" << std::endl;


double Kp = 1.2345;
double Ki = 0.5678;
double Kd = 0.0912;

std::printf("CALCULATED PID VALUES (Placeholder):\n");
std::printf("Kp = %.5f\n", Kp);
std::printf("Ki = %.5f\n", Ki);
std::printf("Kd = %.5f\n", Kd);
std::printf("-----------------------------------------\n");

return 0;
```

```cpp
int main() {
    // ---------------------------------------------------
    // Parameters
    double M = 3.5;          // Mass of Cart [kg]
    double m = 1.49;          // Mass of Pendulum Rod [kg]
    double L = 0.505668;     // Length to Center of Mass [m]
    double r = 0.0355;       // Wheel Radius [m]
    double g = 9.80665;      // Gravity [m/s^2]
    double I = ;            // Moment of Inertia

    // 1. System Matrices
    double den = I * (M + m) + M * m * std::pow(L, 2);

    MatrixXd A(4, 4);
    A << 0, 1, 0, 0,
         0, 0, -(std::pow(m, 2) * g * std::pow(L, 2)) / den, 0,
         0, 0, 0, 1,
         0, 0, (m * g * L * (M + m)) / den, 0;

    MatrixXd B(4, 1);
    B << 0,
         (m * std::pow(L, 2) + I) / (den * r),
         0,
         -(m * L) / (den * r);

    // 2. LQR Cost Weights
    MatrixXd Q = VectorXd::Constant(4, 0).asDiagonal();
    Q.diagonal() << 0.1, 0.1, 500, 1;

    MatrixXd R(1, 1);
    R << 1;

    // 3. Calculate Optimal Gains (K)
    MatrixXd K = solveLQR(A, B, Q, R);
```

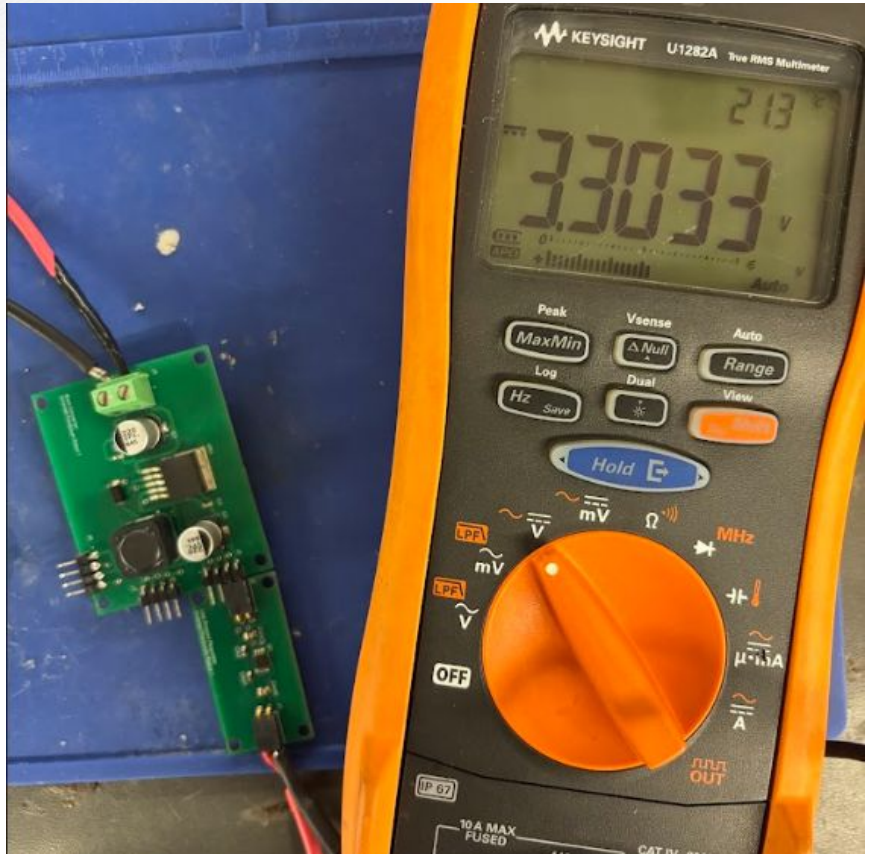
Power & Mechanical

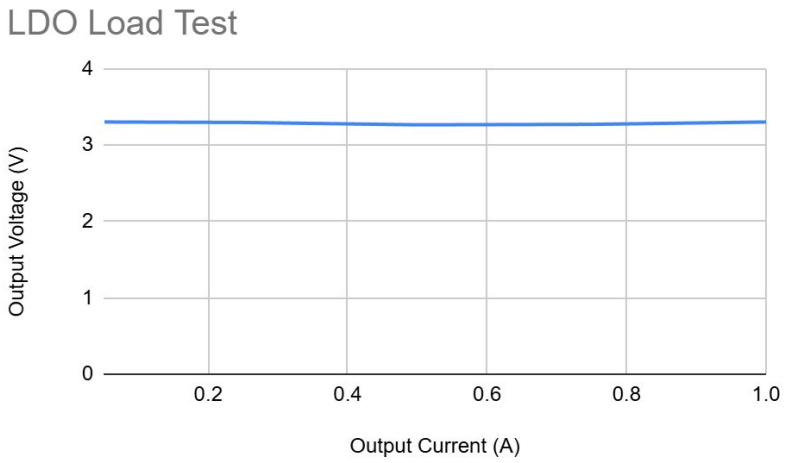
Kaitlynn Ly

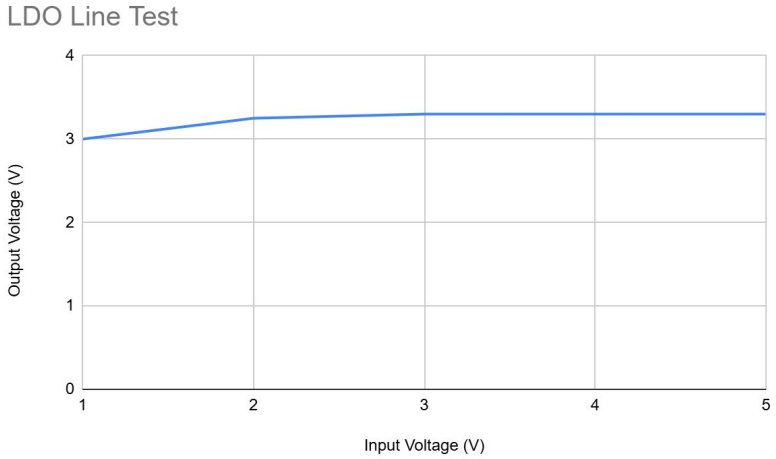
Accomplishments since last update 11 hrs of effort	Ongoing progress/problems and plans until the next presentation
• LDO PCB testing • Pendulum fabrication • Integrated fuse and power switch with battery; tested converter and motor functionality	• Integration with the MCU • Pendulum mounting

Power & Mechanical

Kaitlynn Ly

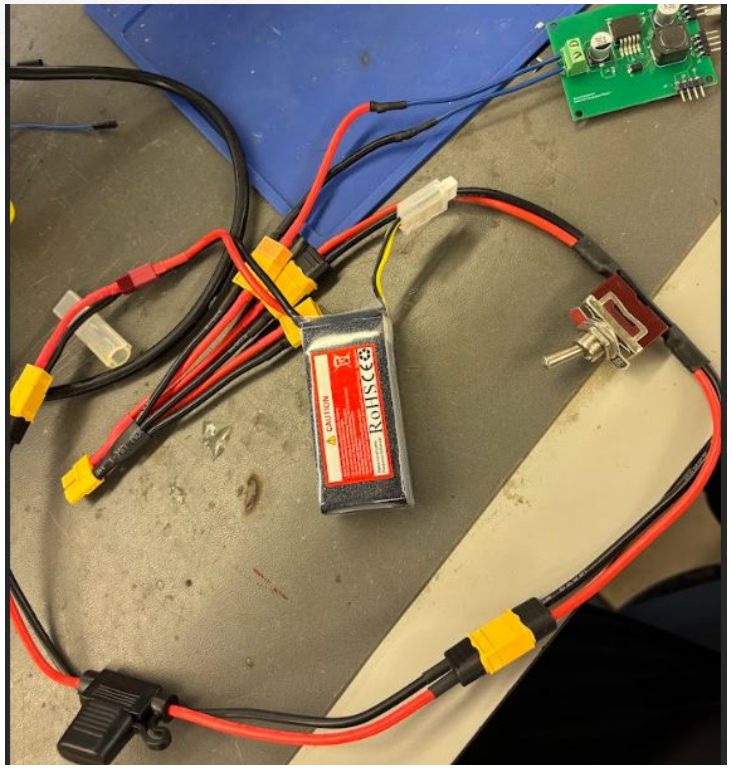




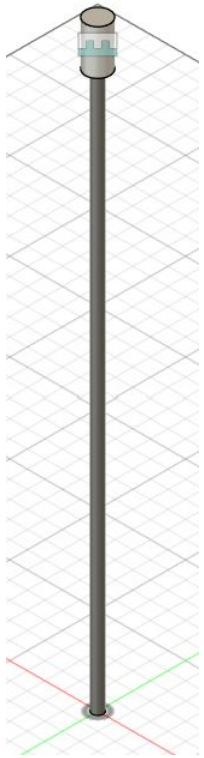


Power & Mechanical

Kaitlynn Ly



Power Input Protection and Regulation



Pendulum Design

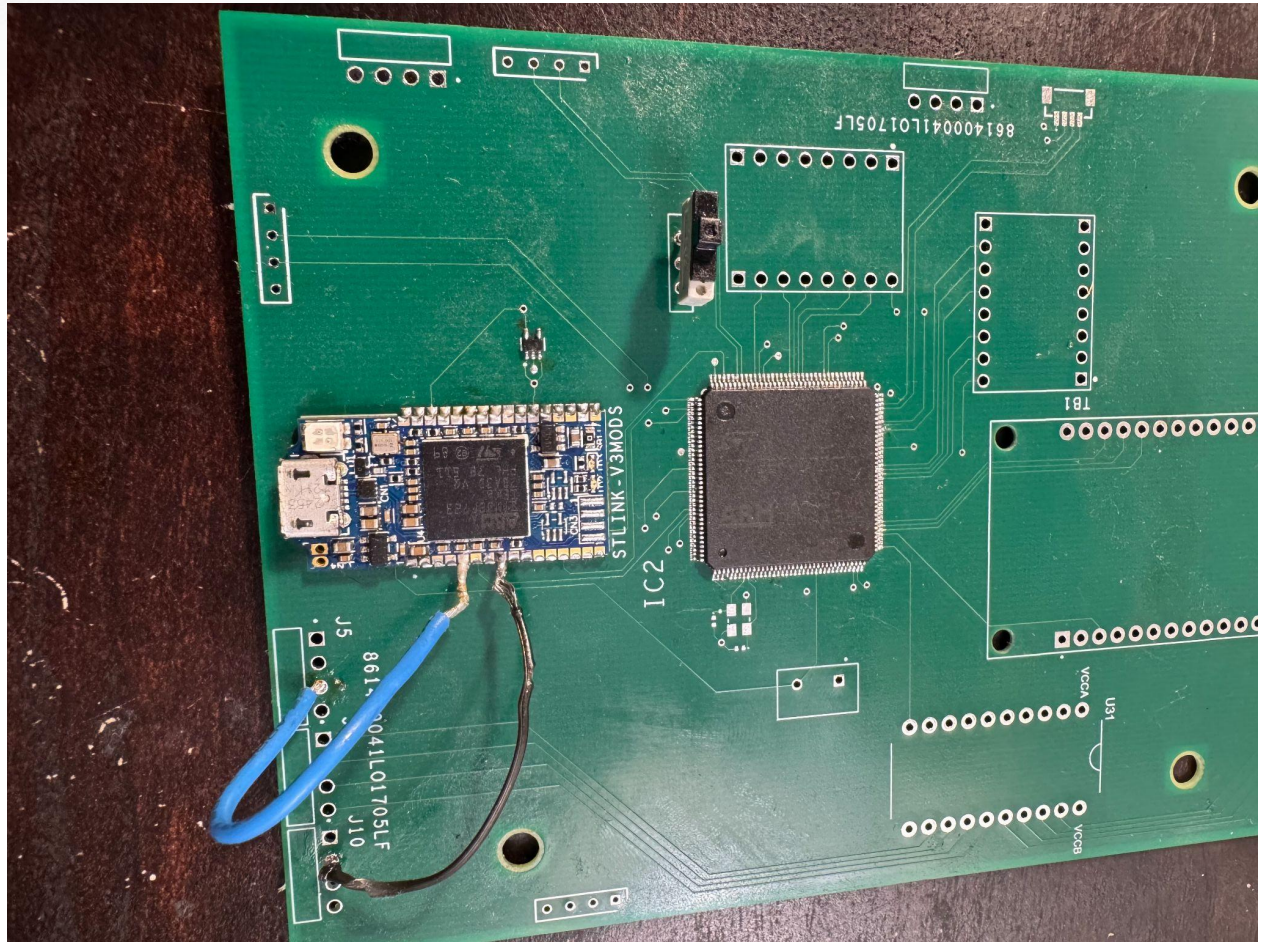
Microcontroller and Logic PCB

Gage Fox

Accomplishments since last update 18 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Implementation of STLINKV3-MODS/programming circuit. • Verification of PCB functionality (few errors, need to be addressed in new Revision). • Isolation of shorting problem on board. • Purchase of temporary Development Board	• Integrate other subsystems together using development board. • Determine shorting issue (suspected damaged STM32) • Manufacture PCB Rev. 2 • Transition to PCB Rev. 2

Microcontroller

Gage Fox



Programming Circuit on PCB Rev 1

Sensors & Navigation

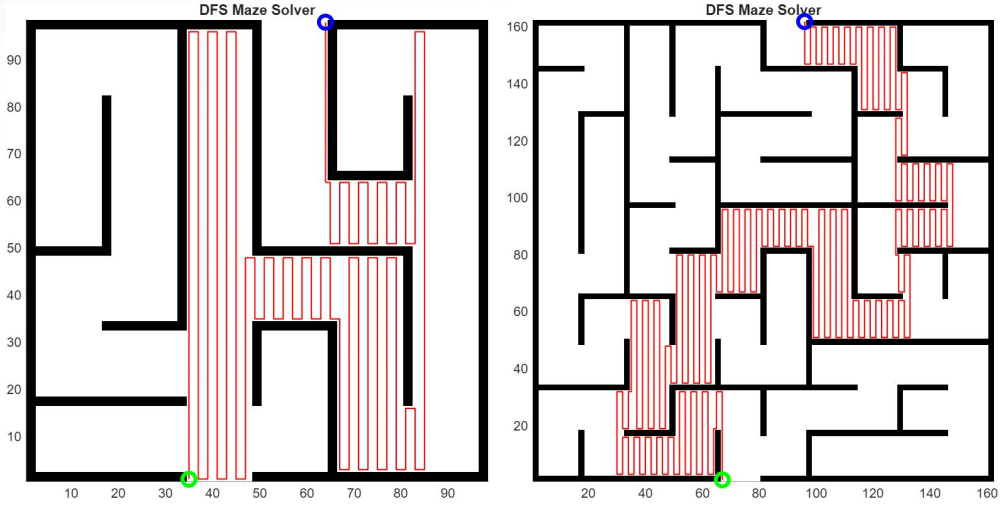
Eric Chang

Accomplishments since last update 15 hrs of effort	Ongoing progress/problems and plans until the next presentation

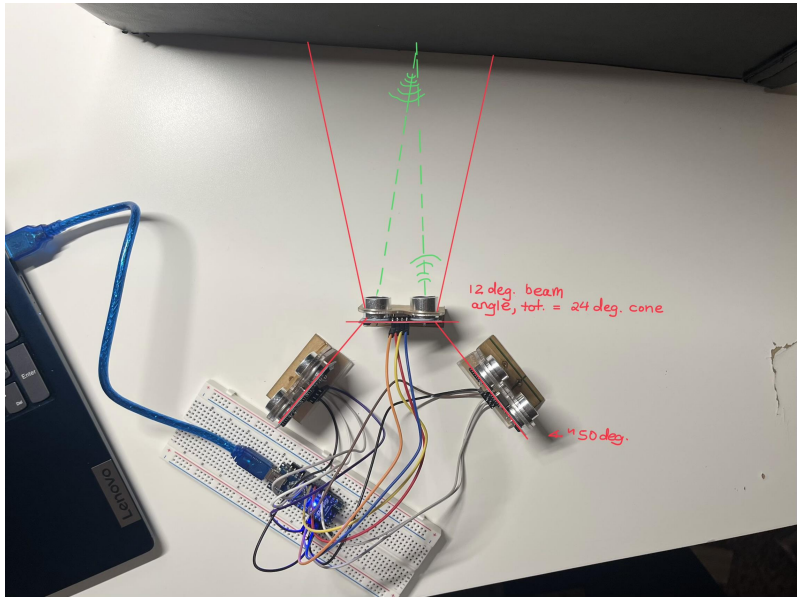
• Ultrasonic sensor sequential firing and calibration tuning
  • Serial plotter to examine and limit cross-talk (interference)
• IMU angle estimation
• Maze solving logic using depth-first search
	• Validate time delays between hardware
	• Sensor fusion integrated on software level, pending on-robot validation
	• Limited ability to test real motion effects off-robot
	  • Mounting and wiring sensors

Sensors & Navigation

Eric Chang



Maze algorithm using depth-first search approach (MATLAB)



Cross-talk validation layout, 50deg diff. between front-facing and side sensors

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Execution Plan

WBS NUMBER	TASK TITLE	TASK OWNER	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	Subsystem Finalization and Integration Planning					
1.1	Schedule integration milestones	All	1/15/26	1/15/26	0	100%
1.2	Review system architecture	All	1/15/26	1/18/26	3	70%
1.3	Review hardware compatibility	All	1/15/26	1/18/26	3	70%
1.4	Define system interface	All	1/15/26	1/18/26	3	100%
1.5	Estimator & GUI prototyping	Eric C.	1/15/26	1/22/26	7	40%
1.5.1	Finalize interface layout with reliable data	Eric C.	1/15/26	1/18/26	3	80%
1.5.2	Implement data logging	Eric C.	1/15/26	1/20/26	5	20%
1.5.3	Prototype extended kalman filter on recorded data	Eric C.	1/20/26	1/25/26	5	20%
1.5.4	Sensor cross-talk validation	Eric C.	1/26/26	1/29/26	3	100%
1.6	Define connector pinouts	Gage F.	1/15/26	1/18/26	3	100%
1.6.1	Solder Components to Logic PCB	Gage F.	2/16/26	2/17/26	5	0%
1.6.2	Code SPI and I2C Comm. Protocols	Gage F.	2/4/26	2/7/26	3	0%
1.6.3	Test SPI and I2C Comm. Protocols	Gage F.	2/9/26	2/13/26	1	0%
1.6.4	Design and Manufacture Logic PCB Rev.2	Gage F.	1/19/26	02/02/26		80%
1.7	Simulate controller behavior in MATLAB	Moustafa K.	1/15/26	1/18/26	3	100%
1.7.1	Convert Code to C++	Moustafa K.	1/15/26	1/18/26	3	100%
1.8	Finalize power protection scheme	Kaitlynn L.	1/15/26	1/18/26	3	100%
1.9	Validate converter performance under different loads	Kaitlynn L.	1/15/26	1/18/26	3	100%
	Finalize pendulum design and mounting	Kaitlynn L.	1/15/26	2/6/26	5	90%
	Integrate motor drivers and validate power to motors	Kaitlynn L.	1/15/26	2/6/26	2	100%
2	Staged System Integration					
2.1	MCU and Controls Integration	Moustafa K.	9/15/25	9/19/25	4	0%
2.2	Sensor-to-MCU Physical Link	All	9/22/25	10/3/25	11	0%
2.3	Sensor Data received	All	9/24/25	9/24/25	0	0%
2.4	Power hooked on	Kaitlynn L.	9/26/25	10/1/25	5	0%
2.5	Test multiple sensors working simultaneously	Eric C.	2/5/26	2/12/26	7	0%
2.5.1	Measure and limit cross-talk/interference	Eric C.	2/5/26	2/12/26	7	0%
2.6	Integrate estimator output to control loop	Eric C.	2/12/26	2/19/26	7	0%
2.6.1	Test estimator timing between sensors	Eric C.	2/12/26	2/19/26	7	0%
2.7	Implement live fault notifications from subsystems in GUI	Eric C.	2/19/26	2/22/26	3	0%
2.8	Controls Algorithm cleanly reads sensor data	Moustafa K.	9/26/25	9/30/25	4	0%
2.9	Test Single-Component Interfacing with MCU	Gage F.	2/18/26	2/22/26	7	0%
2.9.1	Test simultaneous component interfacing with MCU	Gage F.	2/23/26	2/27/26	4	0%

Execution Plan

3	Performance Tuning and System Validation					
3.1	Power and Mechanical Construction	Kaitlynn L.				
3.1.1	Verify regulated 5V/3.3V rails under MCU load	Kaitlynn L.	2/12/26	2/26/26	3	0
3.1.2	Verify power stability during motor start up and braking	Kaitlynn L.	2/26/26	3/9/26	0	0
3.1.3	System Power Integration	Kaitlynn L.	2/15/26	3/1/26	0	0%
3.1.4	Integration Debug and Rework	Kaitlynn L.	2/1/26	3/15/26	0	0%
3.1.5	Verify full system power on sequence	Kaitlynn L.	2/26/19	3/15/26	0	
3.2	Controls subsystem construction	Moustafa K.				
3.2.1	Controls code modified onto MCU	Moustafa K.	02/10/26	3/1/26	15	0%
3.2.2	Controls and sensors integration	Moustafa K.	02/02/26	3/5/26	16	0%
3.2.3	Verify Sensor output is properly read and executed	Moustafa K.	02/10/26	3/10/26	16	0%
3.2.4	Verified control output on physical robot	Moustafa K.	03/19/26	3/26/26	7	0%
3.2.5	Verify controls and mcu integration	Moustafa K.	03/26/26	4/2/26	6	0%
3.2.6	Subsystem Validation	Moustafa K.	04/02/26	4/9/26	7	0%
3.3	Validate sensor performance during dynamic operation	Eric C.	2/26/26	3/5/26	9	0%
3.3.1	Identify changes in bias, drift, and noise under motion	Eric C.	3/5/26	3/8/26	3	0%
3.3.2	Verify encoder accuracy during movement	Eric C.	3/19/26	3/26/26	7	0%
3.4	Finalize logging for integrated system	Eric C.	3/26/26	3/31/26	5	0%
3.5	Tune estimator and controller gains; robustness testing	Eric C.	4/2/26	4/9/26	7	0%
3.6	Tune all communication protocols	Gage F.	3/1/26	3/4/26	3	0%
3.6.1	Fine-tune DMA protocols and peripheral priority	Gage F.	3/5/26	3/10/26	5	0%
3.7	Implement PWM breaking safety features	Gage F.	3/11/26	3/14/26	3	0%
3.8	Determine and Implement noise reductions	Gage F.	3/16/26	3/20/26	5	0%
3.9	Optimize MCU code	Gage F.	3/23/26	3/27/26	3	0%
4	Demonstration and Reporting					
4.1	Final project demonstration	All	4/20/26	4/20/26	0	0%
4.2	Final report	All	4/27/26	4/27/26	0	0%

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Validation Plan

Task	Specifications and Success Criteria	Methodology	Status	Responsible Engineer(s)
Logic Calculation Rate	Validate through timing tests on MCU execution cycle; must complete calculations in < 50ms	Measured using STLINKV2 Debugger	Tested	Gage
Data Transfer Rates	Measure message latency with time logs; must be < 100 ms difference	Measured using STLINKV2 Debugger and Analog Discovery Kit	Tested	Gage
Oscillator Precision	Validate clock stability through frequency tests; ensure deviation < 10ppm		Untested	Gage
Logic PCB Functionality	Verify that Logic PCB functions as intended without signal interferenece between components	Run test signals through all traces and read the resultant waveforms of all traces using an Analog Discovery Kit	Untested	Gage
I2C Communication	Verify that I2C Communication with IMU is functioning without error	Run the IMU alongside the MCU and compare the MCU's readings with the sensor data GUI.	Untested	Gage
SPI Master-Slave Communication	Verify proper Master-Slave communication between MCU and Wireless Module	Send and recieve test signals between the MCU (master) and Wireless Module (slave) and output these signals to their respective hosts.	Untested	Gage
Simultaneous Peripheral Controls and Comms	Validate that all peripherals operate as intended whilst being operated altogether.	Test run that the motors can be controlled with incoming data from all sensors, all at the same time using the MCU.	Untested	Gage
Sonar Read Distance	Validate using logged ultrasonic data (CSV) and IMU; confirm ≥95% obstacle detection at distances ≥2 cm and ±3 cm accuracy within 0.2-1.0 m	Use a meter stick to track distances, record N amount of trials at different positions, analyze collected results	Tested	Eric
Sensor Field-of-view Testing	Verify that the ultrasonic sensor coverage meets the ±15° design requirements	Use a meter stick to track distances, record N amount of trials at different positions, analyze collected results	Tested	Eric
IMU steady-state: bias, noise, drift	Verify IMU steady-state metrics: accel bias ±0.02 g, gyro bias ±0.5 °/s, accel noise σ ≤ 0.03–0.05 g, gyro noise σ ≤ 0.3–0.5 °/s, drift ≤1 °/min	Hold the IMU steady (without external disturbances) for an extended period, analyze collected results	Tested	Eric
Telemetry/GUI Latency	Mean latency < 200 ms with minimal packet loss	A clock will measure the time delay between the data sent from hardware and data received by the web interface	Tested	Eric
Random-walk Performance	Compare path efficiency ratios (PER) of Lévy, uniform-turn, and Gaussian-turn models to determine the best open-space navigation strategy	Utliize MATLAB to perform several simulations of random-walk methods, repeat this for several trials, calculate PER to determine most efficient model	Tested	Eric
IMU and Sonar Dynamic Measurements	Mean error ≤2 ° during motion, mean distance error ≤ 3 cm	Keep the sensors in motion and compare recorded readings against reference values	Untested	Eric
Multi-sensor Coordination	Verify that sensor readings are ≥90% accurate while sensors are active, readings show minimal interference	Run all sensors simultaneously and check for false readings	Tested	Eric
Logging Test	Ensure the system is capable of logging all data during a complete run	Output sensor and control data to the web interface during motion and verify that data is accurate and timely	Untested	Eric
Motor Power Consumption	Measure motor current at multiple voltages under both free-spin and load conditions; confirm total motor power draw is ~40-45 W	Read motor current under different volatges	Tested	Kaitlynn
Verify Input Power Delivery	Measure system voltage under load; confirm range is within 11.1-12.6V	Use an oscilloscope to measure voltage under different loads	Tested	Kaitlynn
Buck Converter Regulation	Record output voltage under different loads; verify 5V stability	Verify 5V stability under multiple loads using an oscilloscope	Tested	Kaitlynn
LDO Validation	Measure regulated output with oscilloscope; confirm 3.3V stability	Measured 3.3V stability	Tested	Kaitlynn
MCU Power Delivery	Measure 3.3V at MCU during code execution; confirm stable operation	Measure 3.3V at MCU during code execution and confrim stable operation	Tested	Kaitlynn
Sensor Power Stablity	Measure sensor supply while streaming data; confirm no dropouts	Measure sensor supply while streaming data	Untested	Kaitlynn
Motor Driver Power	Measure driver input voltage/current during motor operation	Measure motor current at multiple voltages under both free-spin and load conditions; confirm total motor power draw is ~40-45 W	Untested	Kaitlynn
System Power Sequencing	Confirm safe startup; power logic rails first, then motor power	Confirm safe startup; power logic rails first, then motor power	Untested	Kaitlynn
Control Algorithm Validation	Validate stability across various operating conditions and confirm tuned gains achieve target response time	Verified that the controller was able to successfully balance the inverted pendulum	Tested	Moustafa
Initial Condition Response (LQR)	In simulation, initialize the pendulum at -10° offset (without PID input) and observe the angle v. time graph as it stabilizes	In simulation, initialize the pendulum at -10° offset (without PID input) and observe the angle v. time graph as it stabilizes	Tested	Moustafa
Step Response Tracking (PID)	Validate tracking performance by applying a 1.0 m step input; confirm rise time is < 2.5 s, overshoot is < 10%	Validate tracking performance by applying a 1.0 m step input; confirm rise time is < 2.5 s, overshoot is < 10%	Tested	Moustafa
Control Algorithm and MCU Integration	Able to run with the desired rise time, and give commands to rest of robot	Deploy compiled control loops to the MCU; verify PWM signal generation and command transmission frequency meet timing requirements for the desired rise time.	Untested	Moustafa
Control System and Sensor Integration	Able to cleanly read the data from sensors	Perform real-time polling of IMU and encoder registries; verify the control loop processes filtered data without lag or noise-induced jitter.	Untested	Moustafa
Controls System and Power Integration	Control subsystem successfully gives correct power to the robot to deal with various obstacles	Subject the robot to physical barriers; monitor current spikes and duty cycle adjustments to ensure the power system sustains high-torque demands.	Untested	Moustafa

Thank You!

Inverted Pendulum Robot 1

Any Questions?